AUTOMATIZACIÓN CON N8N Y LLM'S

Proyecto Final – BOT TELEGRAM CLIMA LLM

18 DE MARZO DE 2026

CONTENIDO

1 – DESCRIPCIÓN DEL WORKFLOW 2
2 – DIAGRAMA 2
3 – DATOS TÉCNICOS DEL SISTEMA: 3
3.1. Arquitectura del Workflow 3
3.2 Integraciones externas 3
3.3 Modelos de Lenguaje (LLMs) 3
3.4 Lógica condicional 4
3.5 Procesamiento de datos 4
3.6 Sistema de logging 4
3.7 Manejo de errores 5
3.8 Despliegue 5
3.9. Limitaciones 5
4 – PROMPTS UTILIZADOS 6
5 - CONCLUSIÓN GENERAL 6

1 – DESCRIPCIÓN DEL WORKFLOW

El workflow comienza con un trigger automático mediante Telegram, que se activa al recibir un mensaje del usuario con el nombre de una ciudad.

A continuación, se utiliza una API de geocoding para obtener las coordenadas geográficas correspondientes. Luego, se consulta una API de clima para obtener información meteorológica actual.

Los datos obtenidos son procesados en un nodo Set para estructurarlos en un formato claro (temperatura, humedad, probabilidad de lluvia, etc.).

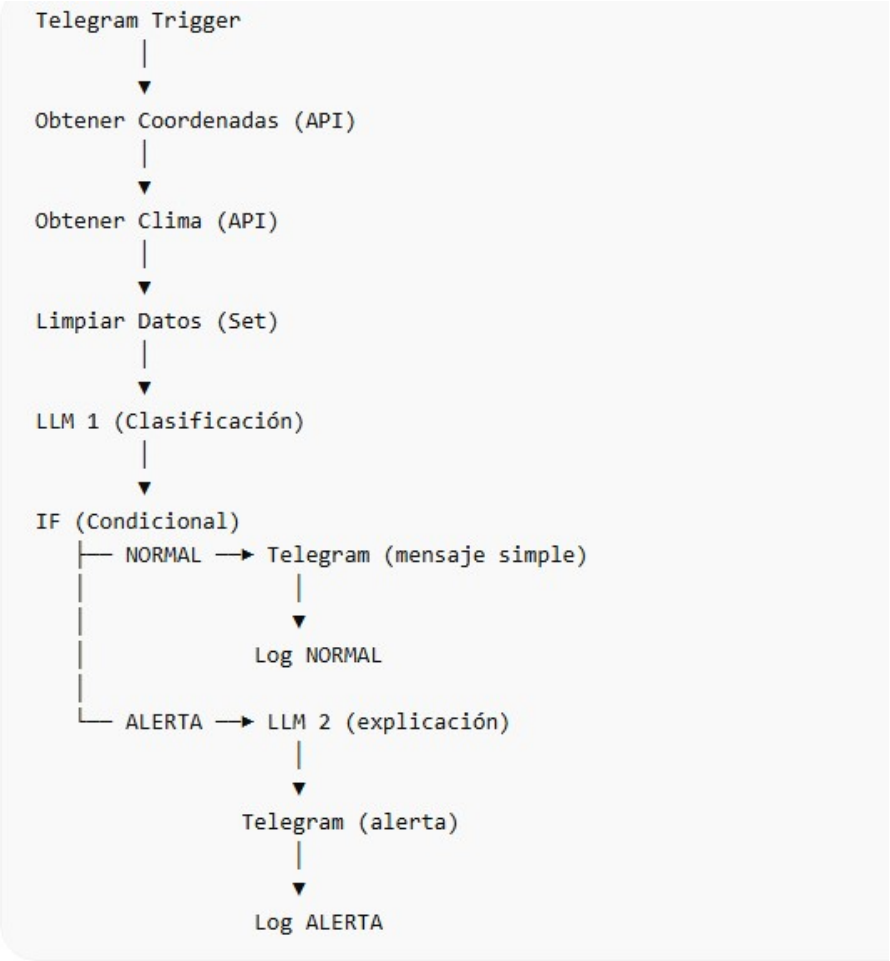
El primer modelo de lenguaje (LLM 1) clasifica el clima en tres categorías: NORMAL, LLUVIA o TORMENTA. Esta salida es utilizada por un nodo IF que determina el flujo de ejecución.

Si se detectan condiciones adversas, se activa un segundo modelo de lenguaje (LLM 2), que genera una alerta con recomendaciones para el usuario.

Finalmente, se envía un mensaje a través de Telegram con la información correspondiente.

El sistema también incluye nodos de logging que registran cada ejecución, permitiendo analizar el comportamiento del workflow y detectar posibles errores.

2 – DIAGRAMA



3 – DATOS TÉCNICOS DEL SISTEMA:

3.1. Arquitectura del Workflow

El sistema fue desarrollado utilizando n8n como herramienta de automatización, implementando un flujo basado en eventos (event-driven architecture).

El workflow se compone de los siguientes módulos:

- Trigger (Telegram)
- Procesamiento de entrada
- Consumo de APIs externas
- Transformación de datos
- Integración con modelos de lenguaje (LLMs)
- Lógica condicional
- Notificación al usuario
- Sistema de logging

3.2 Integraciones externas

Se utilizaron las siguientes APIs:

- Nominatim (OpenStreetMap): Permite convertir el nombre de una ciudad en coordenadas geográficas (latitud y longitud).
- Open-Meteo: Provee datos meteorológicos en tiempo real, incluyendo:
  - Temperatura
  - Humedad
  - Probabilidad de lluvia
  - Precipitación acumulada
- Telegram Bot API: Utilizada para recibir input del usuario y enviar respuestas automatizadas.

3.3 Modelos de Lenguaje (LLMs)

Se integraron dos modelos LLM utilizando Groq:

<u>LLM 1 — Clasificador</u>

- Modelo: Llama 3 (vía Groq)
- Función: Clasificación del clima
- Output: NORMAL / LLUVIA / TORMENTA
- Tipo de salida: texto estructurado (controlado por prompt)

LLM 2 — Generador

- Modelo: Llama 3 (vía Groq)
- Función: Generación de alertas meteorológicas
- Output: mensaje en formato HTML compatible con Telegram

3.4 Lógica condicional

Se implementó un nodo IF que evalúa condiciones meteorológicas críticas:

- Probabilidad de lluvia > 60%
- Humedad > 80%
- Precipitación diaria > 10 mm

Si alguna condición es verdadera → se activa la rama de alerta.

3.5 Procesamiento de datos

Se utilizó un nodo Set para:

- Filtrar datos relevantes de la API
- Renombrar variables
- Simplificar la estructura JSON
- Preparar los datos para los LLMs

3.6 Sistema de logging

Se implementaron nodos Code para registrar:

- Ciudad consultada
- Tipo de respuesta (NORMAL / ALERTA)
- Timestamp de ejecución
- Mensaje enviado

Esto permite trazabilidad y debugging del sistema.

3.7 Manejo de errores

El workflow contempla:

- Validación de respuestas de APIs
- Control de valores undefined
- Uso de datos simulados para testing
- Posibilidad de extensión con reintentos

3.8 Despliegue

El sistema se ejecuta en entorno local utilizando n8n.

Para exponer el webhook de Telegram se utilizó ngrok, permitiendo recibir eventos externos en entorno de desarrollo.

3.9. Limitaciones

- Dependencia de APIs externas
- Posibles errores en geocoding de ciudades ambiguas
- Variabilidad en respuestas de LLM
- No se implementó persistencia de datos histórica

4 – PROMPTS UTILIZADOS

LLM 1:

Clasifica el clima en una sola palabra:

NORMAL, LLUVIA o TORMENTA.

Responde solo una palabra en mayúsculas.

LLM 2:

Eres un experto en meteorología y protección civil.

Genera alertas claras, breves y útiles.

Usa emojis.

Si no hay riesgo, responde: SIN ALERTA

5 - CONCLUSIÓN GENERAL

El desarrollo de este proyecto permitió implementar un sistema automatizado completo utilizando n8n, integrando múltiples tecnologías y modelos de lenguaje para resolver un caso real de consulta meteorológica en tiempo real.

Se logró diseñar un workflow robusto basado en eventos, capaz de recibir entradas desde Telegram, procesar información mediante APIs externas y tomar decisiones dinámicas utilizando modelos de lenguaje (LLMs). La separación de roles entre un modelo clasificador y otro generador permitió estructurar correctamente la lógica del sistema y garantizar respuestas coherentes y útiles para el usuario.

También, la implementación de lógica condicional basada en datos y resultados de LLMs demostró la capacidad de orquestar flujos inteligentes, adaptando el comportamiento del sistema según distintos escenarios climáticos.

El uso de formatos estructurados y HTML mejoró la presentación de los mensajes, mientras que la incorporación de logs permitió agregar trazabilidad y facilitar el análisis del funcionamiento del sistema.

Si bien el sistema presenta algunas limitaciones inherentes al uso de APIs externas y modelos de lenguaje, se diseñó considerando buenas prácticas de desarrollo, modularidad y escalabilidad, dejando abierta la posibilidad de futuras mejoras como persistencia de datos, integración con nuevas fuentes o ampliación de funcionalidades.

En conclusión, el proyecto cumple con los objetivos planteados, demostrando la capacidad de integrar automatización, inteligencia artificial y lógica condicional en un flujo funcional, eficiente y aplicable a escenarios reales.